

● SYSTEM STATUS: OPERATIONAL

NSC ICT Service Desk & Asset Management System

Database Design, Connectivity & System Architecture Report



STUDENT NAME

[Student Name]

MATRIC NUMBER

[Matric Number]

DEPARTMENT

[Department]

INSTITUTION

[Institution]

Prototype academic project — fictional data only

What We'll Cover

A tour of the system, its database, and how the pieces connect



System Overview

What the application does and who uses it



Architecture & Connectivity

How the frontend, backend, and database talk to each other



Security Measures

How data and accounts are protected



Technology Stack

The frameworks and libraries behind the system



Database Design & ERD

Tables, relationships, and normalization



Testing & Verification

How the system was proven to work

What The System Is

A full-stack prototype web application **that tracks ICT hardware assets and manages staff support requests** for an ICT department in a shipping-related organization.

Every dashboard figure is calculated live from PostgreSQL — nothing is hard-coded.

FOUR ROLE TIERS

Administrator

ICT Officer

Technician

Staff / User



Asset Management



Service Desk



Maintenance Records



Staff Management



Departments



Audit Log & Reports

Technology Stack

A deliberately simple stack — no build tools required to run the frontend



Frontend

HTML5, CSS3, vanilla JavaScript — plus Chart.js for the reports dashboard. No React, no build step.



Backend

Node.js with Express, structured as a REST API — one route module per feature area.



Database

PostgreSQL — a normalized relational schema with constraints enforced at the database layer.

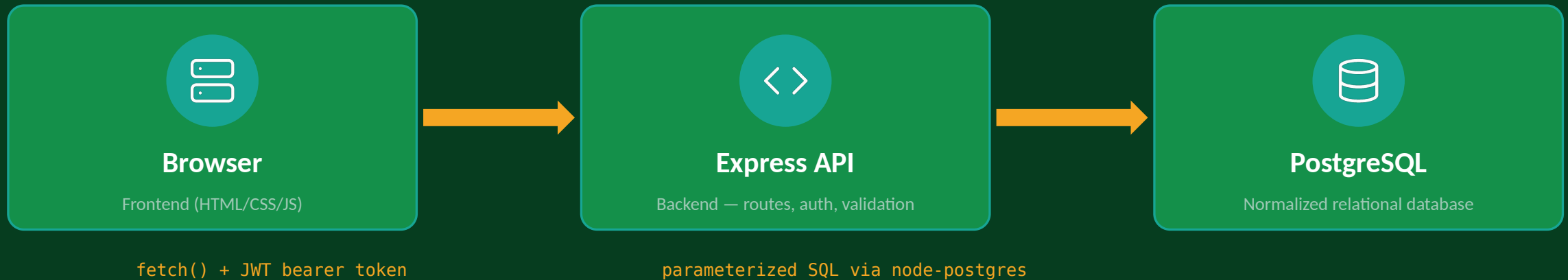


Auth & Security

JSON Web Tokens for sessions, bcrypt for password hashing, role-based middleware on every route.

How The Pieces Connect

A classic three-tier architecture — the frontend never touches PostgreSQL directly



Every write also logs an entry to the `audit_logs` table before the response is sent.

Administrator

Full access

ICT Officer

Manage assets & requests

Technician

Own assigned requests

Staff / User

Submit requests, view assets

Database Connectivity

config/db.js — one shared connection pool, reused by every route

```
const { Pool } = require('pg');

const pool = new Pool({
  host: process.env.PGHOST,
  port: process.env.PGPORT,
  database: process.env.PGDATABASE,
  user: process.env.PGUSER,
  password: process.env.PGPASSWORD,
});

module.exports = pool;

// In a route file:
await pool.query(
  'SELECT * FROM assets WHERE status = $1',
  [status] // parameterized – never
);        // string-concatenated SQL
```



Connection pooling

One Pool instance is shared across every request — no per-query connect/disconnect overhead.



Credentials via environment

Host, database, user, and password are read from .env — never hard-coded, never committed.



Parameterized queries only

Every query uses \$1/\$2 placeholders, which node-postgres binds safely — this is what prevents SQL injection.

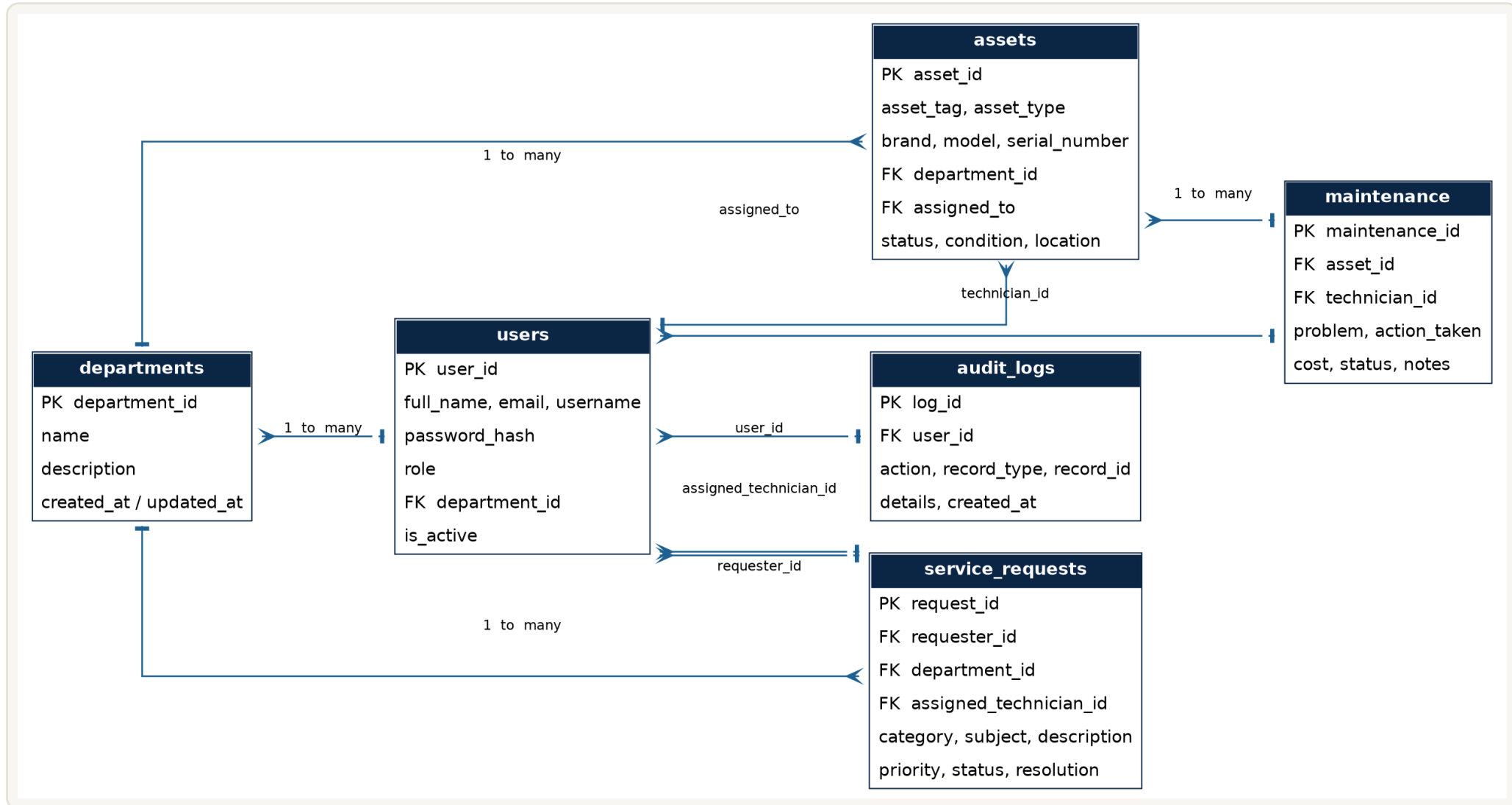


Audit logging on writes

State-changing routes call a shared logAction() helper that inserts into audit_logs after each change.

Entity Relationship Diagram

Matches database/schema.sql exactly — six tables, seven foreign-key relationships



Database Tables

Every column, constraint, and index lives in database/schema.sql

Table	Purpose	Key Relationships
departments	Organizational departments	Referenced by users, assets, service_requests
users	All accounts — 4 roles, bcrypt-hashed passwords	FK to departments
assets	ICT hardware inventory and current status	FK to departments, users (assigned_to)
service_requests	Staff support tickets, end to end	FK to users (requester + technician)
maintenance	Repair / service history per asset	FK to assets, users (technician)
audit_logs	Append-only log of significant actions	FK to users

Security Measures

Enforced at the database and API layer — not just hidden buttons in the UI



Password Hashing

Every password is hashed with bcrypt before storage — plaintext is never saved or logged.



JWT Sessions

Signed, expiring tokens carry the user's role; the signing secret lives only in a local .env file.



Parameterized Queries

All SQL uses \$1/\$2 placeholders via node-postgres — user input is never concatenated into SQL.



Role-Based Middleware

requireAuth and requireRole re-check permissions on the server for every restricted route.



Full Audit Trail

Logins, creates, edits, assignments, and status changes are all written to audit_logs.



Account Deactivation

Deactivated accounts are blocked at login — even with the correct password.

● SYSTEM STATUS: OPERATIONAL

Every asset tracked. Every ticket accounted for.

The ICT department's console for hardware, maintenance, and support requests — one log, one source of truth, always current.

4

ROLE TIERS

6

ASSET STATES

24/7

AUDIT TRAIL

● LIVE DEMO — click "Log In"



NSC ICT Service Desk

ASSET MANAGEMENT SYSTEM

Sign in to your account

Email or Username

e.g. admin or admin@nscict.local

Password

.....

Log In

Don't have an account? [Register here](#)

Demo login — username: **admin** · password:
Password123!



Dashboard

Asset Management

Service Desk

Maintenance

Staff Management

Departments

Reports

Audit Log

About the System

ADMINISTRATOR

Dashboard

Adaeze Okonkwo [Log out](#)

Welcome back, Adaeze Okonkwo. Here is a live snapshot of the system.

Asset Overview

TOTAL ASSETS
20

ACTIVE ASSETS
6

AVAILABLE ASSETS
4

UNDER MAINTENANCE
2

DAMAGED ASSETS
2

RETIRED ASSETS
1

Service Desk Overview

TOTAL REQUESTS
8

PENDING REQUESTS
2

● LIVE DEMO – click a sidebar link

Asset Management

Search tag, brand, model

TAG	TYPE
NSC-LAP-001	Laptop
NSC-LAP-002	Laptop
NSC-DSK-001	Desktop
NSC-PRN-003	Printer
NSC-DSK-004	Desktop
NSC-SRV-001	Server
NSC-DSK-003	Desktop
NSC-PRJ-001	Projector

Add Asset

Asset Tag *

e.g. NSC-LAP-021

Asset Type *

Laptop

Brand

e.g. Dell

Model

e.g. Latitude 5420

Serial Number

Department

— None —

Assigned Staff

— Unassigned —

Purchase Date

Condition

Good

Status

Available

Location

+ Add Asset

CONDITION ACTIONS

ned	Good	History · Edit · Delete
ned	Good	History · Edit · Delete
e	Good	History · Edit · Delete
ged	Poor	History · Edit · Delete
able	Fair	History · Edit · Delete
e	Good	History · Edit · Delete
r Maintenance	Poor	History · Edit · Delete
ed	Poor	History · Edit · Delete

All Statuses ▾ All Priorities ▾

[+ New Request](#)

ID	SUBJECT	REQUESTER	CATEGORY	PRIORITY	TECHNICIAN	STATUS	SUBMITTED	ACTIONS
#1	Laptop not powering on	Ngozi Umeh	Hardware	• High	Chinedu Obi	• In Progress	10 Aug 2026	Assign
#2	Printer paper jam	Tunde Bakare	Printer	• Medium	Fatima Bello	• Assigned	11 Aug 2026	Assign
#3	No internet access	Blessing Nnamdi	Network	• Critical	—	• Pending	12 Aug 2026	Assign
#4	Email not syncing	Emeka Nwosu	Software	• Medium	Segun Adewale	• Resolved	03 Aug 2026	Assign

Purpose of the System

The NSC ICT Service Desk and Asset Management System is a prototype web application built to help an ICT department track its hardware assets and manage staff-reported technical support requests from a single, centralized platform. It is developed as an academic Computer Science / SIWES project and uses entirely fictional sample data.

Problems the System Addresses

- Manual, paper- or spreadsheet-based tracking of ICT equipment, which is error-prone and hard to search.
- No centralized record of who is using which asset, or an asset's maintenance history.
- Ad-hoc, undocumented handling of staff IT complaints with no accountability trail.
- Lack of visibility into technician workload and resolution performance.
- No audit trail of who changed what and when.

Main Features

- Secure, role-based authentication (Administrator, ICT Officer, Technician, Staff/User)
- Live dashboard with statistics computed directly from the database
- Full asset lifecycle management: add, edit, assign, reassign, status changes, history
- Service desk ticketing with categories, priorities, and statuses
- Dedicated technician workspace for managing assigned tickets
- Maintenance records linked to individual assets
- Staff and department management
- System-wide audit logging
- Reports with charts and CSV export

Technologies Used

- **Frontend:** HTML5, CSS3, vanilla JavaScript (no build step required), Chart.js for charts
- **Backend:** Node.js with Express.js (REST API)
- **Authentication:** JSON Web Tokens (JWT) + bcrypt password hashing
- **Database:** PostgreSQL (relational, normalized schema)

Database Used

PostgreSQL. The full schema, relationships, and sample data are provided as SQL files in the project's `/database` folder, and documented in detail in the separate Database Documentation report.

Project Objectives

- **LIVE DEMO** – click a sidebar link

- Build a complete, working full-stack CRUD application connected to a real relational database.
- Apply role-based access control to a multi-user business workflow.

Testing & Verification

70 / 70

AUTOMATED INTEGRATION TESTS PASSING

Run against the real, unmodified backend source — over real HTTP requests.

WHAT WAS COVERED

- ✓ Authentication — login, registration, deactivated-account logout, invalid credentials
- ✓ Role-based permissions — every restricted endpoint checked for every role
- ✓ Full asset lifecycle — create, search, filter, edit, assign, status change, delete
- ✓ Service desk workflow — submission through assignment to resolution
- ✓ Maintenance records — creation, status transitions, asset status sync
- ✓ Staff & department management, audit logging, and report/CSV export

Conclusion & Recommendations

CONCLUSION

The database models the ICT department's core operations — assets, staff, requests, maintenance, and an audit trail — in a normalized, referentially-sound schema. Every dashboard statistic is computed live, and the frontend-to-database integration was verified end to end through parameterized queries and role-checked middleware.

RECOMMENDATIONS

- Add SSL and tuned connection pooling before production use
- Introduce scheduled backups (`pg_dump`) once real data is in use
- Add refresh tokens and login rate-limiting to the auth layer
- Consider lookup tables if enumerated fields need to grow dynamically

6

DATABASE TABLES

4

ROLE TIERS

70/70

TESTS PASSING

22

REPORT SECTIONS

● END OF TRANSMISSION

Thank You

Questions on the system, the schema, or the connectivity are welcome.

